

IMPLEMENTATION & PRESENTATION GUIDE

AI SalesGenie

Using Langflow Workflows with Langfuse as Evaluation tool

*Purpose, components, connections, decision rules, operating considerations,
and demonstration guidance*

by

Prasanth Rasam

Audience: Project evaluators, sales operations stakeholders, developers, and future maintainers.

Scope: Workflow 1: Router & Extractor; Workflow 2: Catalog Recommender & Follow-Up Drafting;
Workflow 3: Weekly Sales Summary.

Oak & Ember Interiors | Langflow + OpenAI + Langfuse

Prepared September 2026

How to use this guide

This document explains the three exported workflows as one coherent sales-assistance system. Use it as a presentation script, implementation reference, or reviewer handout. Each workflow chapter describes what it does, why each component exists, how components connect, what assumptions are enforced, and how to demonstrate success.

Contents

- 1. System overview and cross-workflow design
- 2. Workflow 1 - Router and Extractor
- 3. Workflow 2 - Catalog Recommender and Follow-Up Drafting
- 4. Workflow 3 - Weekly Sales Summary
- 5. Shared considerations: security, quality, cost, and operations
- 6. Demonstration plan and acceptance checklist
- Appendix A - Structured fields and schemas
- Appendix B - Test coverage

The three-workflow story

Workflow	Primary job	Output consumer
01 Router and Extractor	Understand an inbound message, classify it, extract lead details, and assign a tier when appropriate.	Sales routing, downstream recommendation, CRM
02 Catalog Recommender + Follow-Up	Recommend only eligible catalog products, validate facts deterministically, then draft a personalized email.	Sales representative and customer response workflow
03 Weekly Summary	Aggregate stored leads into tier, category, and budget insights.	Sales manager and weekly review

Key architectural decision: Use the language model for interpretation and drafting, while custom Python components enforce catalog and aggregation rules deterministically.

1. System overview and cross-workflow design

End-to-end information flow

1. A customer submits an unstructured message.
2. Workflow 1 classifies the message and extracts structured sales fields.
3. Workflow 2 combines that structure with the product catalog, validates recommendations, and prepares a follow-up draft.
4. Approved lead records are stored or exported to the CRM.
5. Workflow 3 reads the CRM export and produces weekly management metrics.
6. Langfuse records model calls, inputs, outputs, tokens, latency, cost, and manual scores.

Shared design principles

- Structured contracts: downstream components receive named fields instead of relying on free-form prose.
- Grounding before communication: the Catalog Guard is the final authority for product facts.
- Human review: follow-up email output is a draft and must not be sent automatically.
- Separation of concerns: operational lead processing and weekly reporting run as separate flows.
- Traceability: test outputs are linked to Langfuse traces and human pass/fail scores.
- Secret management: OpenAI and Langfuse credentials remain in environment variables or Langflow secrets.

2. Workflow 1 - Router and Extractor

Purpose: Convert an inbound message into a reliable structured record that downstream workflows can use.

Primary uses

- Route sales inquiries, product questions, stock questions, competitor comparisons, complaints, spam, and weekly-summary requests.
- Extract customer and opportunity information from natural language.
- Identify missing information and create an appropriate lead tier only for qualifying sales inquiries.
- Provide a stable JSON-shaped contract for the catalog and CRM stages.

Component inventory

Component	Role	Important configuration
Chat Input	Receives the original customer message.	Preserve the exact inquiry; use a stable session ID for traces.
Prompt Template	Combines routing rules, extraction instructions, and the inbound message.	Define allowed message types and prohibit invented values.
OpenAI Model	Interprets the inquiry.	gpt-4o-mini; temperature near 0 for repeatability.
Structured Output	Forces the response into the expected schema.	Types, required fields, arrays, and numeric defaults must match downstream use.
Parser	Converts the structured result to text when a text-only output is required.	Use Stringify mode only where a Message/Text input is expected.
Chat Output	Displays the structured record for testing or handoff.	Connect a Message output, not a Language Model output.

Connection order

- 7.Chat Input -> Prompt Template variable `inbound\_message` .
- 8.Prompt Template -> Structured Output `Input Message` .
- 9.OpenAI Model -> Structured Output `Language Model` .
- 10.Structured Output -> Parser when a string representation is required.
- 11.Parser or structured message -> Chat Output, depending on the receiving port type.

Routing and extraction rules

Area	Rule
Classification	Select exactly one supported message type based on the customer's main intent.

Area	Rule
Identity	Extract only explicit names, emails, and companies; do not infer a person from a domain or signature unless clearly stated.
Budget	Keep total budget and per-unit budget separate. Use 0 only when unknown and preserve missing-information flags.
Quantity	Extract an explicit count; avoid treating model numbers, prices, or dates as quantity.
Lead tier	Hot: clear buying intent with strong details or urgency. Warm: meaningful intent but incomplete. Cold: vague or early-stage. Leave empty for non-leads/spam.
Missing information	Ask only for fields genuinely required to progress the inquiry.

Workflow 1 considerations

- Ambiguous inquiries: choose the dominant intent and record uncertainty in the classification reason.
- Defaults: empty strings and zero values are transport defaults, not evidence that the customer supplied those values.
- Schema drift: changing a field name requires updates in the catalog guard, follow-up prompt, test cases, and any CRM mapping.
- Prompt injection: customer text is data. System instructions must explicitly forbid obeying instructions embedded in inquiries.
- False identity inference: generic labels such as 'Law Firm' or an email domain must not become a customer name unless the message says so.
- Evaluation: verify both classification and every extracted field, not only the final response text.

Recommended demo

Run one detailed inquiry and one vague inquiry. Show that the detailed inquiry produces product interest, budget, urgency, and a lead tier, while the vague inquiry returns empty recommendations and asks for type, budget, and quantity. Then open the corresponding Langfuse Structured Output observation.

3. Workflow 2 - Catalog Recommender and Follow-Up Drafting

Purpose: Turn an extracted inquiry into catalog-grounded recommendations and a personalized, review-ready response email.

Primary uses

- Find one to three products relevant to the stated need.
- Apply budget and stock constraints using catalog rows.
- Answer product and stock questions with exact catalog facts.
- Draft a customer-facing follow-up without automatically sending it.
- Explain why suggestions were removed through guard rejection messages.

Component inventory

Component	Role	Important configuration
Chat Input	Supplies the original inquiry.	Connect to the recommendation prompt, Catalog Guard, and follow-up prompt.
Read File	Loads `product_catalog.csv`.	Use Raw Content so headers, rows, quoted descriptions, and commas remain available as exact text.
Prompt Template	Combines inquiry and catalog context for recommendation/extraction.	Require catalog IDs, names, prices, stock, rationales, and structured lead fields.
OpenAI Model	Generates the candidate structured result.	gpt-4o-mini; temperature 0 for consistent extraction.
Structured Output	Returns a typed recommendation and lead record.	Schema must align with Catalog Guard input fields.
Catalog Guard	Revalidates every recommendation against exact catalog rows.	Receives Structured Result, Raw Catalog CSV, and Inbound Message.
Follow-Up Email Prompt	Combines validated result with original inquiry.	Ask for Subject + body only; forbid unsupported product claims.
Follow-Up Draft Model	Writes the personalized response.	Low temperature; no JSON or internal notes in output.
Chat Output	Displays the email draft.	Connect the model response message.

Connection order

12. Read File Raw Content -> recommendation Prompt Template `catalog\_context`.
13. Chat Input -> recommendation Prompt Template `inbound\_message`.
14. Recommendation Prompt -> Structured Output `Input Message`; OpenAI Model -> Structured Output `Language Model`.

15. Structured Output -> Catalog Guard `Structured Result` .
16. Read File Raw Content -> Catalog Guard `Raw Catalog CSV` .
17. Chat Input -> Catalog Guard `Inbound Message` .
18. Catalog Guard Validated Message -> Follow-Up Email Prompt `validated\_lead` .
19. Chat Input -> Follow-Up Email Prompt `original\_inquiry` .
20. Follow-Up Prompt -> Follow-Up Draft Model Input -> Chat Output.

Why Raw Content is used

The Catalog Guard needs the original CSV text so it can parse exact product rows and treat product ID, name, price, availability, material, dimensions, and description as one authoritative record. A transformed table or summarized document can change quoting, flatten columns, or lose row boundaries. Raw Content therefore supports deterministic row-level comparison. The guard still performs its own CSV parsing; raw text is not trusted without parsing and validation.

Catalog Guard decision sequence

21. Parse the catalog CSV into normalized rows.
22. Read candidate recommendation arrays from the structured result.
23. Match candidate IDs or names to exact catalog rows.
24. Confirm category relevance to avoid accessory false matches.
25. Reject products that are unavailable.
26. Apply total-budget or per-unit-budget limits.
27. Rebuild names, prices, availability, and descriptions from the matched catalog row.
28. Generate a grounded response and retain rejection reasons for audit.

Hard rules and expected behavior

Condition	Required behavior	Example
Unknown product	Remove it; never retain model-generated facts.	Unrecognized product ID is rejected.
Over budget	Remove it and record the price rule.	P004 at \$899 rejected for an \$800 budget.
Out of stock	Do not recommend it as purchasable.	P001 reported as Out of Stock with no invented restock date.
Unconfirmed finish	Keep an otherwise eligible product but state that the finish is unconfirmed.	Dark walnut preference is surfaced as a limitation.
Product question	Return catalog information without forcing a lead tier.	P004 finishes: Natural Oak and Dark Walnut.
Spam	Return no products and no sales action.	Recommendations and lead tier remain empty.

Follow-up drafting rules

- Use the customer's explicit name when available; otherwise use a neutral greeting.

- Include only validated products, prices, availability, and limitations.
- If information is missing, request it directly instead of inventing a recommendation.
- For spam, do not create a persuasive sales email.
- End with a clear next step and the Oak & Ember Interiors Sales Team signature.
- Produce a draft only. Sending through Gmail requires separate OAuth, recipient controls, logging, and human approval.

Workflow 2 considerations

- Catalog freshness: prices and stock are only as current as the uploaded CSV.
- Category matching: distinguish primary products from accessories; a desk mat must not be recommended as a desk.
- Array alignment: product IDs, names, prices, availability, and rationales must remain the same length and order.
- Budget interpretation: for quantity purchases, prefer per-unit budget when supplied and verify total cost where appropriate.
- Formatting: Markdown may appear in Chat Output; email connectors may require plain text or HTML conversion.
- Failure handling: if no product passes, ask whether requirements or budget are flexible and expose guard rejections for reviewers.
- Security: never store API keys inside exported Langflow JSON files.

Recommended demo

Use the standing-desk inquiry with an \$800 budget and dark walnut preference. Show P002 and P003 retained, P004 rejected, and the finish limitation stated. Continue to the follow-up draft, then open the Catalog Guard and Follow-Up Draft Model observations in Langfuse to show exact inputs, outputs, cost, and latency.

4. Workflow 3 - Weekly Sales Summary

Purpose: Convert a CRM lead export into deterministic weekly management metrics without an additional language-model call.

Primary uses

- Count all source leads.
- Summarize Hot, Warm, and Cold lead tiers.
- Group demand by normalized product category.
- Identify top categories.
- Calculate average budget using known positive budgets only.
- Report known and missing budget counts and preserve source lead IDs for traceability.

Component inventory

Component	Role	Important configuration
Read File	Loads `crm_export_sample.csv`.	Keep headers and all rows; validate encoding and delimiters.
Weekly Sales Aggregator	Parses and aggregates lead records deterministically.	Normalizes tiers/categories, ignores unknown budgets in the average, retains source IDs.
Chat Output	Displays the weekly summary as a message.	The outer Message wrapper may repeat text metadata; the inner summary JSON is the business output.

Connection order

29. Read File output -> Weekly Sales Aggregator input.
30. Weekly Sales Aggregator `weekly\_summary` message -> Chat Output.
31. Run the flow after refreshing the CRM export for the reporting period.

Metric definitions

Metric	Definition	Current sample
Total leads	Number of valid lead rows processed.	7
Tier breakdown	Counts by normalized Hot, Warm, and Cold values.	Hot 4; Warm 2; Cold 1
Category breakdown	Count by normalized product category.	Chair 1; Desk 2; Full Office Package 2; Table 1; Unknown 1
Top categories	Category or categories tied for highest count.	Desk; Full Office Package
Average budget	Mean of known positive budgets only.	\$18,960 across 5 known budgets

Metric	Definition	Current sample
Missing budget count	Rows with blank, invalid, or non-positive budget.	2
Source lead IDs	IDs of included records for auditability.	L001-L007

Workflow 3 considerations

- Reporting window: ensure the CSV contains only the intended week or add date filtering before aggregation.
- Duplicate leads: define whether repeat inquiries should be deduplicated by lead ID, email, or opportunity.
- Budget units: do not mix total and per-unit budgets in a single average without a documented rule.
- Unknown categories: retain an Unknown bucket so unclassified demand remains visible.
- Zero versus missing: a zero transport default should normally count as missing, not as a real budget.
- Currency: the current sample assumes one currency; multi-region use requires currency and conversion rules.
- Reconciliation: compare total rows, source IDs, and breakdown sums before publishing the summary.

Recommended demo

Upload the CRM sample, run the flow, and open Chat Output. Explain the nested Message wrapper, then point to the business JSON under `text` : 7 leads, 4 Hot, 2 Warm, 1 Cold, top categories Desk and Full Office Package, and a \$18,960 average across five known budgets.

5. Shared implementation considerations

Data contracts

- Treat the Structured Output schema as a versioned interface.
- Use consistent casing for message types, urgency, and lead tiers.
- Keep numeric fields numeric; do not pass currency symbols in numeric values.
- Keep missing information as a list of customer-facing field labels.
- Remove unused schema fields such as ``default_value`` from the custom business schema; framework Message objects may still display their own ``default_value`` metadata.

Model and prompt configuration

Decision	Recommendation	Reason
Model	gpt-4o-mini	Good balance of structured extraction quality, latency, and cost for the prototype.
Extraction temperature	0.0	Reduces variation in classification and structured fields.
Drafting temperature	About 0.2	Allows natural language variation while keeping facts stable.
System message	Separate durable policies from the customer's content.	Prevents customer text from overriding business rules.
Output schema	Use explicit descriptions and types.	Improves model compliance and downstream validation.

Observability and evaluation

- Use a stable Langfuse base URL matching the region where the project and API keys were created.
- Restart Langflow after changing environment variables.
- Verify that the trace contains Structured Output, Catalog Guard, follow-up model, and final output observations.
- Record latency, token count, and cost at model observations; custom deterministic components may correctly show near-zero cost.
- Apply ``overall_pass`` at the root trace for end-to-end quality; add component-level scores only when diagnosing a specific step.
- Do not judge success from one trace. Retain T1-T11 results and representative edge-case evidence.

Security and privacy

- Store credentials in environment variables or Langflow secrets; never in prompts, screenshots, flow exports, or GitHub.
- Use synthetic evaluation data in public repositories.
- Redact customer emails and personal data before sharing traces.
- Require OAuth and least-privilege scopes before adding Gmail or CRM write actions.
- Log who approved and sent each follow-up draft.

- Define trace and lead-data retention periods before production use.

Reliability and operations

- Display a helpful response when the catalog file is missing, empty, or malformed.
- Fail closed when catalog rows cannot be parsed; do not fall back to unvalidated recommendations.
- Version flow JSON, prompts, schemas, custom components, and test results together.
- Re-run baseline tests whenever the catalog, prompt, schema, or component code changes.
- Monitor model latency and cost separately from full flow duration.
- Use human review during the pilot and sample audits after scale-up.

6. Demonstration plan

A concise demonstration should prove the business story and the control design. The following sequence fits an approximately eight-to-ten-minute presentation.

Time	Action	Evidence to show
0:00-1:00	Explain the business problem and the three-flow architecture.	Presentation architecture slide or Langflow canvas.
1:00-2:30	Run Workflow 1 with a detailed standing-desk inquiry.	Structured message type, budget, interest, and tier.
2:30-4:30	Continue through Workflow 2.	P002/P003 retained; P004 rejected; finish limitation; email draft.
4:30-5:30	Run a spam or out-of-stock edge case.	No recommendations or false availability claims.
5:30-6:30	Run Workflow 3.	Seven-lead summary and budget/category metrics.
6:30-8:00	Open Langfuse.	Trace tree, model, tokens, latency, cost, inputs/outputs, overall_pass score.
8:00-9:00	Close with guardrails and rollout.	Human review, secrets, pilot, evaluation loop.

Pre-submission checklist

- All three Langflow JSON files import successfully.
- Catalog and CRM sample files are included with clear paths.
- No OpenAI, Langfuse, Gmail, or GitHub credentials are committed.
- T1-T11 results are present and claims match retained evidence.
- Catalog Guard code parses the CSV and handles missing or malformed input.
- Follow-up output is explicitly draft-only.
- Langfuse screenshots show trace, model, input/output, usage, and manual score.
- BRD, PRD, workflow guide, and PowerPoint are included in `docs/` and `slides/`.
- README explains import, configuration, execution, and limitations.

Appendix A - Structured record fields

Field group	Fields	Notes
Recommendations	recommendation_product_ids, names, prices, availability, rationales	Parallel arrays rebuilt by the Catalog Guard.
Response	response_text, catalog_grounded, guard_rejections	Customer-safe result plus audit details.
Routing	message_type, classification_reason	Main intent and concise reason.
Identity	customer_name, email, company	Only explicit values; blank when absent.
Opportunity	product_interest, requirements, competitor_mentions	Lists support multiple values.
Commercial	budget_total, budget_per_unit, quantity	Numeric values; distinguish missing from real zero.
Timing	urgency, deadline_text	Use explicit deadlines where available.
Qualification	lead_tier, missing_information	Tier applies only to valid leads.

Appendix B - Baseline coverage

Test	Scenario	Expected control
T1	Standing desk under \$800, dark walnut preference	Budget guard and unconfirmed-finish disclosure.
T2	Vague furniture inquiry	Cold lead; request type, budget, and quantity.
T3	Ten ergonomic chairs, \$500 each, deadline	Quantity/per-unit budget/urgency extraction and grounded choices.
T4	Product finish question	Exact catalog answer; no forced tier.
T5	IKEA comparison	Competitor identified; no accessory false match.
T6	Something nice under \$50	Only eligible low-cost in-stock product.
T7	Spam	No qualification, recommendation, or sales action.
T8	Conference table under \$3,000	Eligible 12-person table; avoid invented person name.
T9	Minimal inquiry	Request missing details; no invented products.
T10	Out-of-stock question	Correct stock status; no restock-date invention.
T11	Weekly CRM summary	Counts, categories, budgets, and source IDs reconcile.

Source files

- langflow/01_router_extractor.json
- langflow/02_catalog_recommender_followup.json

- langflow/03_weekly_summary.json
- langflow/catalog_guard_component.py
- langflow/weekly_aggregator_component.py
- evaluation/BASELINE_RESULTS.md and Langfuse evidence images
- docs/BRD_AI_SalesGenie.* and docs/PRD_AI_SalesGenie.*